Rapport Technique : Moteur d'Orchestration d'Opérations Omnicanal

1. Architecture retenue et organisation du projet

Pour que le code soit propre et modulaire, j'ai découpé le projet comme ça:

orchestrator/ :

- base.py : le modèle de base avec la méthode run() qui gère les étapes.
- mixins.py : les options réutilisables à la carte (logs, cache, retry, audit, horodatage).
- operations.py : les opérations métier (CSV, API, stats, rapport) et ComplexOperation.
- plots.py : la génération des graphiques Matplotlib (barres, courbes, nuages).
- exceptions.py : mes erreurs personnalisées (ValidationError, TransientError, etc.).
- main.py : le fichier qui lance les pipelines et exécute les opérations.

tests/ :

- test_operations.py : teste si les opérations marchent avec des données valides et invalides.
- test_cache.py : vérifie que le résultat est bien mémorisé au 2ème appel.
- test_retry.py : vérifie que ça réessaie bien en cas d'erreur temporaire.
- test_plots.py : teste les graphiques sans ouvrir de fenêtre.
- test_mro.py : vérifie que l'ordre d'héritage (MRO) est respecté.

La colonne vertébrale du projet :

Pour partager les variables communes sans copier-coller de code, j'ai mis en place une chaîne d'héritage :
CooperativeBase -> BaseOperation -> IOOperation -> DataOperation.

- CooperativeBase : sert de base commune et gère le bon fonctionnement des appels avec super().
- BaseOperation : donne le modèle de base avec la méthode run() pour toutes les opérations.
- IOOperation : gère la source et la destination (sink) pour les entrées/sorties.
- DataOperation : gère les données en mémoire via l'attribut data.

Les opérations finales (comme CSVImportOperation ou BarPlotOperation) héritent de cette base et piochent dans les mixins selon ce dont elles ont besoin.

2. Le rôle des mixins et le cycle de vie

Pour ajouter des options sans alourdir la classe de base, j'ai créé 7 mixins :

-LoggedMixin : affiche les étapes dans la console.

-AuditedMixin : garde la trace des étapes dans la liste audit_trail

-TimestampedMixin : enregistre la date de création et met à jour la date d'exécution.

-ValidatedMixin : permet de lancer une ValidationError si les données sont fausses.

-CachedMixin : mémorise le résultat pour ne pas recalculer au 2ème appel.

-RetryMixin : relance automatiquement l'opération en cas d'erreur temporaire (TransientError)

-SerializableMixin : transforme l'opération en dictionnaire avec to_dict()

Dans BaseOperation, la méthode run() fixe un ordre strict (patron Template Method)

validation, exécution du code métier dans do_run(), puis envoi du résultat. Les classes filles ne modifient jamais run(), elles codent seulement leur logique dans do_run().

3. Le polymorphisme

Dans mon moteur (main.py), toutes les opérations s'exécutent avec la même commande :

```
for operation in operations:

   resultat = operation.run()

   print(f"Type : {type(operation).__name__} | Résultat : {resultat}")
```

4. Le fonctionnement de super()

Dans mes constructeurs et mes mixins, j'appelle systématiquement super().__init__(*args, **kwargs). En héritage multiple, super() ne saute pas forcément au parent du dessus : il passe la main à la classe suivante dans l'ordre du MRO. *args et **kwargs permettent de faire passer les arguments inconnus d'une classe à l'autre sans bloquer la chaîne.

5. La résolution du losange

Le losange apparaît quand deux mixins différents héritent de la même classe mère CooperativeBase. Pour ne pas exécuter l'ancêtre commun deux fois, Python utilise l'algorithme C3 pour ranger toutes les classes dans une liste unique sans doublon. En haut, CooperativeBase sert de bouchon : elle absorbe les arguments restants pour appeler object proprement.

6. Le MRO de ComplexOperation

En affichant ComplexOperation.mro(), Python donne cet ordre linéaire : ComplexOperation -> les mixins déclarés (CachedMixin, RetryMixin, etc.) -> DataOperation -> IOOperation -> BaseOperation -> CooperativeBase -> object. Grâce à cet ordre, chaque classe passe la main à la suivante sans jamais casser la chaîne.

7. Le fonctionnement du cache

CachedMixin utilise la méthode cache_key(). La clé doit obligatoirement être un type immuable (comme une chaîne ou un tuple) pour être hachable et servir de clé dans le dictionnaire interne du cache. Si la clé existe déjà, le résultat est renvoyé directement depuis la mémoire sans relancer le calcul dans do_run().

8. Le fonctionnement du retry

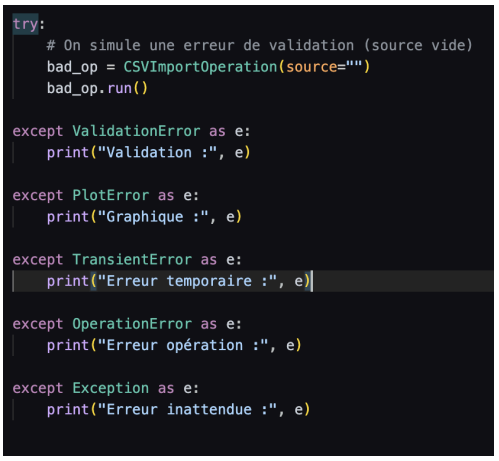
RetryMixin capture uniquement les erreurs temporaires (TransientError). Si le réseau ou l'API échoue, il recommence l'exécution jusqu'au nombre maximal de tentatives configuré. Les autres erreurs, comme une erreur de données (ValidationError), ne déclenchent aucun réessai.

9. L'intégration de Matplotlib

Pour créer des graphiques, j'ai créé une classe intermédiaire PlotOperation qui hérite de DataOperation et des mixins nécessaires (validation, logs, audit, horodatage). Ensuite, chaque type de graphique a sa classe dédiée : BarPlotOperation, LinePlotOperation, ScatterPlotOperation. Chaque graphique vérifie ses données (colonnes X et Y existantes, données non vides) et trace la figure dans do_run() sans ouvrir de fenêtre bloquante pendant les tests.

10. La gestion des exceptions

J'ai créé une hiérarchie d'exceptions dérivant de OperationError (ValidationError, TransientError, PlotError, SerializationError). Dans le code d'exécution (main.py) et les tests, je place toujours les erreurs spécifiques d'abord et Exception tout à la fin :



```python
try:
    # On simule une erreur de validation (source vide)
    bad_op = CSVImportOperation(source="")
    bad_op.run()

except ValidationError as e:
    print("Validation :", e)

except PlotError as e:
    print("Graphique :", e)

except TransientError as e:
    print("Erreur temporaire :", e)

except OperationError as e:
    print("Erreur opération :", e)

except Exception as e:
    print("Erreur inattendue :", e)
```

Si except Exception était placé en premier, il intercepterait tout aveuglément et empêcherait le RetryMixin de capturer les TransientError.

11. Les choix importants réalisés

1. Ordre des mixins : j'ai placé CachedMixin avant RetryMixin. De cette façon, on vérifie d'abord si la donnée est déjà en mémoire. Si on inversait, une panne relancerait des essais réseau inutiles alors que le résultat était déjà prêt en cache.
2. Patron Template Method : la méthode run() est verrouillée dans BaseOperation pour garantir que la validation et l'audit s'exécutent toujours sur chaque opération.
3. Choix entre héritage et composition pour Matplotlib : j'ai choisi l'héritage direct (PlotOperation -> BarPlotOperation) pour respecter les contraintes du TP. En entreprise, une composition avec un objet Renderer serait plus modulaire pour changer de librairie graphique sans toucher aux opérations.

12. Questions de synthèse

1. Qu'est-ce que l'héritage multiple ?

C'est quand une classe fille hérite de plusieurs classes mères en même temps pour combiner leurs méthodes et leurs variables.

2. Qu'est-ce qu'un mixin ?

C'est une petite classe secondaire qui apporte une option réutilisable précise (comme des logs ou du cache) sans avoir vocation à être créée toute seule.

3. Qu'est-ce que le MRO ?

Le MRO (Method Resolution Order) est l'ordre linéaire calculé par Python pour chercher les méthodes et exécuter les classes parentes.

4. Pourquoi l'ordre des parents est-il important ?

Parce que Python lit les classes de gauche à droite. La première classe écrite est prioritaire dans le MRO et interceptera les appels avant les autres.

5. Que fait réellement super() ?

super() ne saute pas obligatoirement au parent direct : il appelle la méthode de la classe suivante dans la liste ordonnée du MRO.

6. Pourquoi utiliser *args et **kwargs dans certains constructeurs ?

Pour que le constructeur accepte n'importe quel paramètre et le transmette à la classe suivante via super().__init__(*args, **kwargs) sans bloquer la chaîne.

7. Comment Python gère-t-il un héritage en losange ?

Il utilise l'algorithme C3 pour ranger les classes dans une liste unique où chaque classe parente commune n'est exécutée qu'une seule fois.

8. Qu'est-ce que le polymorphisme ?

C'est le fait de pouvoir exécuter des objets de classes différentes avec le même appel (comme op.run()) sans tester leur type avec isinstance().

9. Quelle différence existe-t-il entre polymorphisme et héritage ?

L'héritage permet de partager du code d'une classe mère vers ses filles. Le polymorphisme permet d'utiliser des objets différents via un même contrat d'appel.

10. Quel est le rôle d'une classe abstraite ?

Elle définit un contrat obligatoire (via le module abc) en imposant des méthodes abstraites que les classes filles doivent obligatoirement implémenter.

11. Pourquoi créer des exceptions personnalisées ?

Pour distinguer précisément la nature des erreurs (erreur de données avec ValidationError, coupure temporaire avec TransientError) et adapter la réaction du programme.

12. Pourquoi except Exception doit-il généralement être placé après les exceptions spécifiques ?

Parce que Python teste les erreurs dans l'ordre d'écriture. Si on met Exception en premier, il capture tout et rend les blocs spécifiques inaccessibles.

13. Comment fonctionne RetryMixin ?

Il encadre l'exécution dans une boucle. S'il intercepte une TransientError, il recommence l'opération jusqu'à atteindre le nombre maximal d'essais.

14. Comment fonctionne CachedMixin ?

Il calcule une clé avec cache_key(). Si la clé est déjà dans son dictionnaire, il renvoie directement le résultat en mémoire sans relancer le calcul.

15. Pourquoi l'ordre CachedMixin, RetryMixin peut-il être différent de RetryMixin, CachedMixin ?

Avec le cache en premier, on vérifie d'abord la mémoire. Si on mettait le retry en premier, une panne relancerait des essais réseau inutiles alors que le résultat était déjà en cache.

16. Pourquoi Matplotlib a-t-il été intégré sous forme d'opérations plutôt que directement dans le moteur ?

Pour respecter le principe ouvert/fermé : on peut ajouter plein de nouveaux graphiques sans jamais devoir toucher ou modifier le code du moteur d'orchestration.

17. Comment ajouter une nouvelle opération sans modifier l'orchestrateur ?

Il suffit de créer une nouvelle classe qui hérite de BaseOperation (ou DataOperation), d'ajouter les mixins voulus et d'écrire sa logique dans do_run().

18. Dans quels cas la composition serait-elle préférable à l'héritage multiple ?

Quand les fonctionnalités doivent changer dynamiquement pendant l'exécution (par exemple brancher un objet Renderer différent) ou quand l'arborescence d'héritage devient trop rigide.